

RAPPORT TECHNIQUE : CALL ME MAYBE

RAPPORT TECHNIQUE : ANALYSE DU PIPELINE DE DECODAGE CONTRAINT (CALL ME MAYBE)

1. CLASSE CONSTRAINEDPIPELINEDECODER

Cette classe centralise le contrôle de la génération du LLM en manipulant ses logits (probabilités de sortie) pour forcer le respect de contraintes strictes.

- `__init__(self, model):`

- Fonctionnement : Récupère le chemin du fichier de vocabulaire du modèle, le charge et l'inverse sous forme d'un dictionnaire {ID_Token: Texte_Token}.

- Justification des conditions : Le bloc ``if hasattr(...)`` vérifie la présence de `'get_path_to_vocab_file'` ou `'get_path_to_vocabulary_json'`. C'est une sécurité essentielle pour éviter un crash complet de l'application si l'API publique du SDK venait à être mise à jour ou modifiée.

- `_get_logits(self, input_ids):`

- Fonctionnement : Sollicite le modèle pour obtenir les logits du prochain token.

- Justification des conditions : ``if isinstance(logits[0], (list, tuple))`` détecte si le SDK renvoie les logits de toute la séquence (matrice 2D/3D) ou uniquement du dernier jeton. Si c'est une matrice, le code extrait de façon sécurisée ``logits[-1]``, assurant qu'on manipule toujours une liste plate.

- `decode_one_of_candidates(self, prompt_ids, candidates, max_tokens=32):`

- Fonctionnement : Algorithme de type Finite State Machine (FSM). À chaque étape, il applique un masque (`-math.inf`) sur tous les tokens du vocabulaire qui ne forment pas un préfixe valide des mots candidats restants.

- Justification des conditions :

- * ``if len(candidates) == 1``: Si un seul choix est possible, l'algorithme le

renvoie immédiatement, économisant des appels de jetons inutiles au LLM.

* `any(suf.startswith(clean\_tok) for suf in remaining)` : Assure la contrainte stricte du préfixe. Aucun token hors-liste ne peut être choisi.

- generate_free_string(self, ids, max_tokens=16):

- Fonctionnement : Permet une génération libre mais encadrée.

- Justification des conditions :

* `tok\_str.startswith("<|") or any(c.isdigit() for c in tok\_str)` : Assigne une probabilité nulle aux caractères spéciaux et numériques pour éviter qu'un nombre (venant d'autres arguments) ne s'infilte dans une chaîne pure.

* `

' in tok or '}' in tok` : Arrête immédiatement la génération si le modèle tente de sortir de sa structure (injection de retour à la ligne ou JSON).

2. FONCTION PRINCIPALE MAIN() ET ANGLE MORTS

- Gestion du prompt vide (`if not test\_case.prompt`):

- Justification : Évite d'envoyer une séquence vide à la couche de tokens du LLM, ce qui provoquerait une erreur de dimensions dans le framework de deep learning.

- Routage Pur sans "none" explicite :

- Justification : Sur les modèles légers, mettre "none" dans la liste de choix crée un biais d'ancrage fort (le modèle sur-sélectionne "none" par manque de confiance). On ne fait donc choisir le modèle *que* parmi les fonctions valides.

- Validation Sémantique (Anti-Hallucination) :

- Justification : `if any(kw in prompt\_words for kw in semantic\_keywords)`

Puisque "none" est retiré de la liste fermée, un prompt totalement absurde (ex: "who is ely") forcerait la sélection d'une fonction valide au hasard.

Cette condition croise les mots du prompt avec les mots-clés de la fonction élue. S'il n'y a aucun lien logique, le script écrase le choix et applique

"none" dynamiquement. C'est ce qui sauve "Greet shrek" tout en bloquant le bruit.

- Traitement des Paramètres :

- Booléens : ``rfind("true")`` et ``rfind("false")`` vérifient d'abord de manière déterministe la présence brute des mots pour économiser les ressources de calcul.

- Numériques : ``re.findall(r"-?\d+\.\d*")`` extrait mathématiquement les nombres. L'exclusion ``n not in used_numeric_values`` garantit qu'un même nombre ne sera pas assigné à deux variables différentes (ex: a et b dans une opération).

- Cas Regex complexe : Un bloc ``if`` dédié intercepte sémantiquement les patterns comme "replace X with Y" pour mapper les instructions textuelles complexes.